OnBuch+

SIGNED COURSE • PREMIUM

COMPUTER SCIENCE • UPPER SIXTH | UPPER SIXTH — COMPUTER SCIENCE

CHAPTER 8

CATEGORIES OF SOFTWARE

SUGGESTED TIME: 1 PERIODS

IN THIS CHAPTER

This chapter presents the great families of software found on any computer system and the criteria used to classify them. Students learn to distinguish system software from application software, to identify utility and development tools, and to classify software according to usage rights — skills frequently examined at the GCE Advanced Level.

- By the end of this chapter I can define software and distinguish it from hardware.
- By the end of this chapter I can state the main criteria used to classify software.
- By the end of this chapter I can describe the role of system software and give examples (operating systems, drivers, firmware).
- By the end of this chapter I can describe application software and classify it into general-purpose and special-purpose (bespoke) software.
- By the end of this chapter I can identify utility software and programming/development software and explain their roles.
- By the end of this chapter I can classify software according to usage rights: proprietary, freeware, shareware, open source and public domain.

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

CONTENTS

1. Software: concept and classification criteria
2. System software
3. Application software
4. Classification by usage rights and synthesis
5. Integration activity
6. Methods and worked examples
7. Exercises
8. Solutions to the exercises
9. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can define software and distinguish it from hardware.
- By the end of this chapter I can state the main criteria used to classify software.
- By the end of this chapter I can describe the role of system software and give examples (operating systems, drivers, firmware).
- By the end of this chapter I can describe application software and classify it into general-purpose and special-purpose (bespoke) software.
- By the end of this chapter I can identify utility software and programming/development software and explain their roles.
- By the end of this chapter I can classify software according to usage rights: proprietary, freeware, shareware, open source and public domain.
- By the end of this chapter I can classify any given software into its correct category and justify my choice in an examination context.
- By the end of this chapter I can advise a user on the appropriate category of software for a given need.

PREREQUISITES

- Definition of a computer system: hardware and software components (Lower Sixth).
- Basic notion of an operating system as the program that starts the computer (Lower Sixth).
- Vocabulary: program, data, file, user, licence.

- Everyday experience of using common applications (word processor, browser, media player).

1 SOFTWARE: CONCEPT AND CLASSIFICATION CRITERIA

1.1 From the cybercafé to the classroom: why classify software?

At a cybercafé in Bamenda, a student pays to type her CV on a computer running Windows 11, Microsoft Word, an antivirus, and a small billing program written by a local developer. Why does the computer need Windows before Word can even start? Why is the antivirus always running in the background, and why did the developer's program cost money while the web browser was free? Behind these everyday questions lies a fundamental idea: not all software plays the same role on a computer. This chapter teaches you to recognise, classify and justify the category of any software you meet — a classic GCE Advanced Level question.

The problem we must solve is therefore the following: given any piece of software (Windows, Word, an antivirus, a billing program, a free browser), how can we describe it **precisely** and **rigorously**, so that two computer scientists anywhere in the world understand each other without ambiguity? The answer is a **classification**: a small number of clear criteria that sort every software product into well-defined families.

1.2 Recall: the place of software in a computer system

From Lower Sixth you already know that a **computer system** is made of three inseparable components:

- the **hardware**: the physical, tangible part — the parts you can touch (keyboard, screen, processor, memory, hard disk);
- the **software**: the intangible part — the programs and data that make the hardware work;
- the **users** (sometimes called liveware): the human beings who operate the machine.

Hardware without software is like a generator without fuel and without a driver: powerful, but completely useless. Software is what transforms an inert collection of electronic circuits into a tool that can type a CV, play music, or compute a school's report cards.

DEFINITION **Software**

Software is the set of programs, procedures and data that instruct the hardware of a computer what to do and how to do it. It is the intangible (non-material) part of a computer system.

DEFINITION **Program**

A **program** is an ordered sequence of instructions, written in a programming language, which the computer executes in order to perform a precise task.

Notice the hierarchy: a single instruction ("add these two numbers") is the atom; a program is

an ordered collection of instructions; software is a collection of programs together with the data and documentation they need. Microsoft Word, for instance, is software: it contains many programs (for editing, printing, spell-checking) plus data files (dictionaries, templates).

WARNING **Software ≠ a program**

A very common mistake in the GCE examination is to use "software" and "program" as synonyms. They are not. A **program** is one element; **software** is the whole set of programs, procedures and data. Saying "I installed a software" to mean one small program is imprecise: you installed a program, which is part of the software of the machine. In your answers, always ask yourself: am I talking about one element (program) or about the whole collection (software)?

1.3 Why do we classify software?

Classifying is not a game for examiners. It has concrete, practical purposes:

1. **To understand roles.** Knowing that Windows is system software immediately tells you that the machine cannot function without it, whereas deleting a game does not stop the computer from working.
2. **To choose the right tool.** A business in Douala that needs payroll must know whether a general-purpose spreadsheet is enough or whether special-purpose payroll software is required.
3. **To manage licences and costs.** Before installing anything, a school or company must know whether the software is paid (proprietary) or free/open, because this determines the budget and the legal rights of use, modification and redistribution.

METHOD **The three questions to ask before classifying any software**

Faced with any software product, ask these three questions in order:

1. What role? Does it manage the machine itself (system software), or does it serve a user's task (application software)? Is it perhaps a supporting tool (utility, development tool)?
2. What licence? Must you pay and respect restrictions (proprietary), or is it free to use, study, modify and share (free/open source)?
3. General or specific purpose? Can it serve many different tasks and users (general-purpose), or was it built for one precise task or one specific client (special-purpose/bespoke)?

The three answers together give the complete classification of the product.

1.4 Criterion 1 — Role or function in the system

The first and most important criterion is the **role** the software plays inside the computer system. It divides software into two great families, with two supporting families.

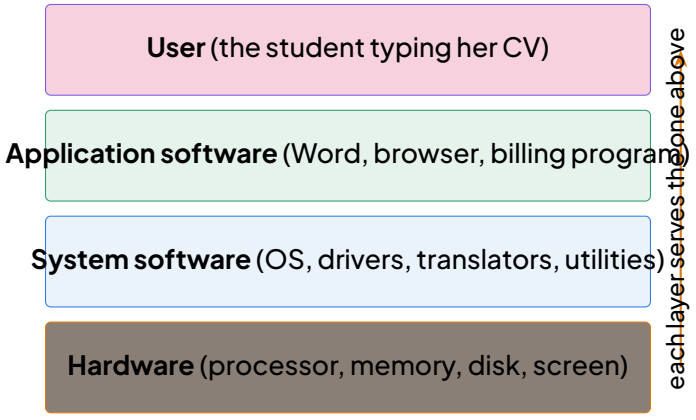
DEFINITION **System software and application software**

System software is software that manages, controls and supports the computer hardware so that other programs can run. Its main representative is the operating system (Windows, Linux, Android). It also includes device drivers and language translators (compilers, interpreters, assemblers). **Application software** is software that performs a task directly useful to the user: typing a letter, browsing the web, calculating a budget.

Between and around these two families stand two supporting families:

- **Utilities** (or utility programs): small service programs that maintain, protect and optimise the system (antivirus, disk cleanup, compression tools, backup tools, file managers). They assist the system software.
- **Development tools** (or programming tools): programs used to create other programs (compilers, interpreters, editors, debuggers, linkers, loaders). They serve the programmer rather than the ordinary end-user.

The relationship between these families and the hardware is best seen as a stack of layers: each layer can only work because the layer below it does its job.



The layered view of a computer system: the user interacts with applications, which rely on the system software, which drives the hardware.

This layered diagram answers the cybercafé question directly: Word cannot start before Windows because Word is an application — it asks the operating system for memory, files and screen space. The antivirus runs permanently in the background because it is a utility: its job is precisely to watch over the system at all times.

1.5 Criterion 2 — Usage rights: the licence

The second criterion concerns the **legal rights** attached to the software, defined by its **licence** (the contract between the producer and the user).

DEFINITION **Proprietary software and free/open source software**

Proprietary software is software whose use is restricted by a licence (usually paid): the user may use it under conditions, but may not see the source code, modify it, or redistribute it. Examples: Microsoft Windows, Microsoft Office, Adobe Photoshop. **Free software** (often called open source software) is software distributed with a licence that grants the freedom to use, study (the source code is available), modify and redistribute it. Examples: GNU/Linux, LibreOffice, Mozilla Firefox, VLC media player.

WARNING **"Free" does not always mean "free of charge"**

Do not confuse free software (freedom to use, study, modify, share) with freeware (distributed at no cost but still closed and restricted). A free browser may cost nothing and be open source; another program may cost nothing yet remain proprietary. In the exam, justify your answer by the rights granted by the licence, not only by the price.

This criterion explains why the local developer's billing program cost money (proprietary licence — it is his work and his income) while the web browser was free to install (open licence).

1.6 Criterion 3 — Distribution of purpose: general or specific

The third criterion looks at **for whom and for what** the software was built.

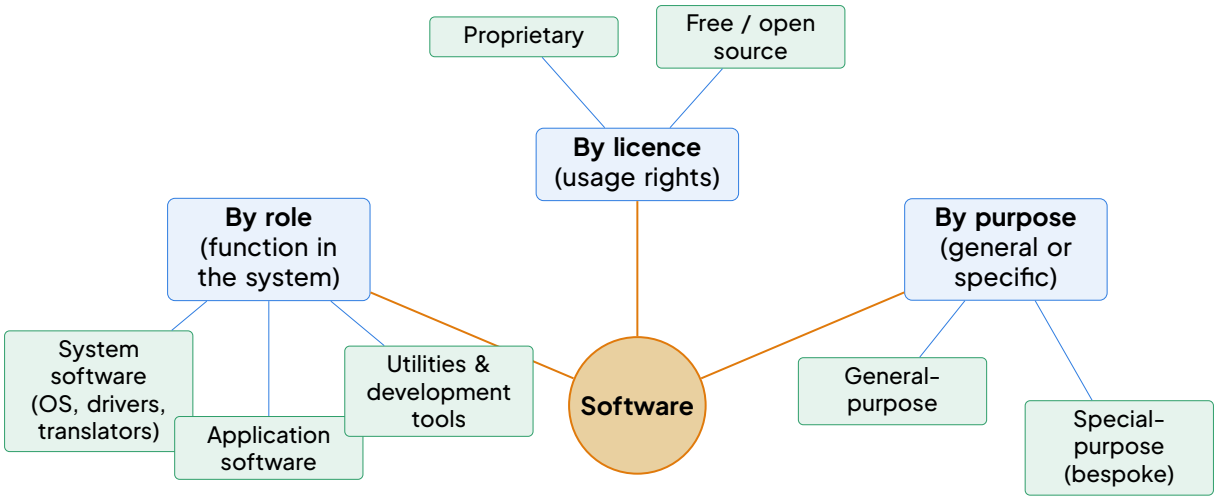
DEFINITION **General-purpose and special-purpose software**

General-purpose software is designed to serve a wide variety of users and tasks: a word processor can type a CV, a letter or a novel; a spreadsheet can handle a budget, marks or stock. It is sold "off the shelf" or downloaded for immediate use. **Special-purpose software** (also called bespoke or customised software) is designed for one specific task or one specific client: the billing program of the Bamenda cybercafé, the examination-results system of a school, the software of an ATM machine, the control system of a factory robot.

General-purpose software is cheaper per user and immediately available, but may not fit a particular need perfectly. Special-purpose software fits the need exactly, but is expensive to develop and maintain, because the cost is not shared among thousands of buyers.

1.7 The complete classification tree

The three criteria are independent: they can be combined freely. The mind map below summarises the classification used throughout this chapter.



Mind map of the three classification criteria for software. Any product can be described by one branch of each criterion.

KEY POINTS **Key points**

- A computer system = hardware + software + users; software is the intangible part.
- Software = set of programs, procedures and data; a program = one ordered sequence of instructions in a programming language.
- Three classification criteria: role (system / application / utilities / development tools), licence (proprietary / free-open source), purpose (general / special).
- The criteria combine: one product has one answer on each axis.

EXEMPLE **Classifying the cybercafé software**

Apply the three questions to the Bamenda cybercafé:

- Windows 11: system software (role); proprietary (licence); general-purpose (purpose).
- Microsoft Word: application software; proprietary; general-purpose.
- The antivirus: utility (supporting the system); proprietary or free depending on the

product; special-purpose (one precise task: protection).

- The billing program: application software; proprietary (paid to the developer); special-purpose (bespoke, written for this cybercafé).
- The free browser: application software; free/open source; general-purpose.

PROPERTY **A product belongs to several classifications at once**

The three criteria are independent dimensions, not rival boxes. One software product is therefore described by one value on each criterion simultaneously. For example, Linux is system software (role) and open source (licence) and general-purpose (it runs servers, phones and desktops). Saying "Linux is open source, so it is not system software" is a logical error: the two statements concern different criteria. (This remark is directly examinable as a justification question.)

WORKED EXAMPLE 1 **Classifying unfamiliar software**

Statement. A hospital in Yaoundé buys (i) a licence of Microsoft Excel for its accountants, (ii) a patient-records program written on demand by a software company specifically for that hospital, and (iii) installs the Ubuntu operating system on its machines. For each product, give its category according to each of the three criteria, justifying briefly.

SOLUTION

Solution. We apply the three questions of the method.

- (i) Microsoft Excel. Role: application software — it performs a user task (calculations, tables). Licence: proprietary — a paid Microsoft licence, source code closed. Purpose: general-purpose — the same spreadsheet serves accountants, teachers and traders.
- (ii) Patient-records program. Role: application software — it serves the hospital staff's task directly. Licence: proprietary — developed and sold under contract. Purpose: special-purpose (bespoke) — written for one client and one precise task; it cannot be bought off the shelf by another user.
- (iii) Ubuntu. Role: system software — it is an operating system that manages the hardware and allows the other programs to run. Licence: free/open source — it may be used, studied, modified and redistributed. Purpose: general-purpose — it can run many kinds of machines and applications.

Remark. Note that (i) and (ii) are both application software, yet they differ on the purpose criterion: this shows why all three criteria are needed for a complete description.

EXERCISE 1 **Classify your own environment**

List five pieces of software you have used this month (on a computer or a smartphone). For each one, answer the three questions of the method (role? licence? general or specific purpose?) and present your answer as a table with three columns, one per criterion. Then identify which of them the machine could not function without, and explain why in two sentences.

SOLUTION **Exercise 1**

A possible answer (yours may differ): WhatsApp — application, proprietary (freeware), general-purpose; Android — system software (operating system), open source at its core, general-purpose; a calculator app — application, free, general-purpose; a school's report-card program — application, proprietary, special-purpose; an antivirus — utility, proprietary, special-purpose. The machine cannot function without the operating system (Android), because as system software it manages the hardware and provides the services (memory, files, screen) that every application needs in order to run. All the other items could be removed and the device would still switch on and work.

DID YOU KNOW? **Software in Cameroon**

Most computers in Cameroonian schools and cybercafés run proprietary system software (Windows), but the open-source alternative Linux is widely used on servers, including in universities and some administrations, because it costs nothing in licence fees and can be studied and adapted freely. The choice between proprietary and open software is not only technical: for a school with a small budget, it can determine whether a computer room of twenty machines is affordable or not. Understanding software categories is therefore also an economic skill.

2 SYSTEM SOFTWARE

In the previous section we saw that software can be classified according to several criteria, the most fundamental being its role in the machine: does it run the computer itself, or does it serve a particular user need? We now study the first great family in depth: **system software**, the invisible workforce without which no computer, tablet or smartphone could do anything at all.

2.1 Definition and role of system software

Imagine a newly purchased computer in a shop in Douala, straight out of its box, with no software installed. The hardware is perfect: processor, memory, disk, screen. Yet the machine can

do nothing useful. It is like a magnificent hotel with no manager, no receptionist and no housekeeping staff: the rooms exist, but nobody can check in. System software is that management staff.

DEFINITION **System software**

System software is the set of programs that manage and control the hardware of a computer and provide a platform on which application software can run. It does not perform a specific user task (like writing a letter); instead, it makes the machine usable and offers common services to all other programs.

The main families of system software are: the **operating system**, **device drivers**, **firmware**, **utility software**, and **translators and development tools**. We study each in turn.

2.2 The operating system (OS)

When you switch on a computer, the first large program that takes control is the operating system. Everything you afterwards do — opening a folder, printing a document, connecting to the Internet — passes through it.

DEFINITION **Operating system**

An **operating system** (OS) is the master program of a computer: it manages the hardware resources (processor, memory, storage, peripherals), provides a platform and common services for application programs, and offers an interface through which the user communicates with the machine.

Main functions of an operating system.

An OS performs five essential management tasks:

- **Managing the processor (CPU):** many programs run "at the same time". The OS shares the processor's time between them (this is called scheduling), switching so fast that the user has the impression of simultaneity.
- **Managing memory:** the OS decides which part of the RAM each program may use, and protects programs from interfering with one another. If a program crashes, the others survive.
- **Managing files:** the OS organises data on disks into files and folders, and controls who may read, write or delete them.
- **Managing peripherals:** the OS receives input from the keyboard and mouse, sends output to the screen and printer, and coordinates all devices through drivers.
- **Managing users:** the OS identifies users (login and password), keeps their settings separate, and controls their access rights.

Examples of operating systems.

Operating system	Typical platform	Remark
Microsoft Windows	Desktop and laptop PCs	Most widespread on personal computers
Ubuntu (a Linux distribution)	Desktops and servers	Free and open-source
macOS	Apple Macintosh computers	Supplied with Apple hardware
Android	Smartphones and tablets	Mobile OS based on the Linux kernel
iOS	iPhone and iPad	Apple's mobile OS

Notice the distinction between **desktop/server operating systems** (Windows, Ubuntu, macOS), designed for keyboards, large screens and long sessions, and **mobile operating systems** (Android, iOS), designed for touch screens, batteries and portability. The same fundamental role is adapted to very different machines.

DID YOU KNOW? **Did you know? The first operating systems**

The very first computers (1940s–1950s) had no OS: programmers scheduled machine time on a wall chart and loaded their programs via punched cards. The first true OS, GM-NAA I/O, appeared in 1956 on an IBM 704. In Cameroon, many secondary schools and universities now use Linux distributions (like Ubuntu) because they are free, secure and run well on older hardware — a practical illustration of how system software choices affect real access to computing.

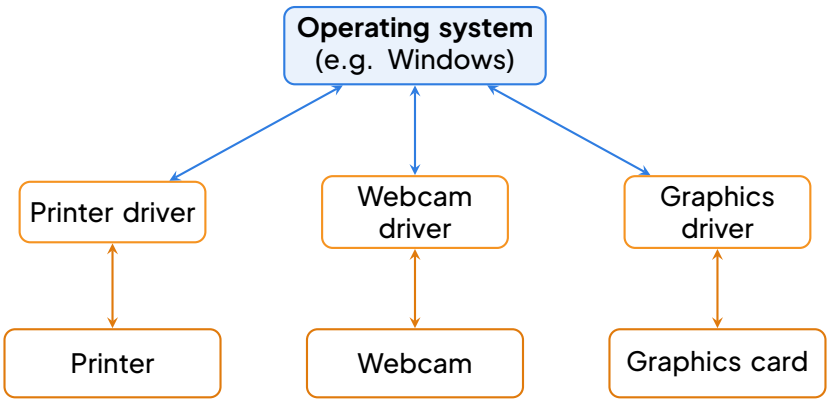
2.3 Device drivers

An operating system is written before most peripherals even exist. How can Windows, released years ago, control a printer manufactured last month in China? The answer is the device driver.

DEFINITION **Device driver**

A **device driver** is a small program that allows the operating system to communicate with a specific peripheral (printer, graphics card, webcam, mouse). It translates the general orders of the OS ("print this page") into the exact commands understood by that particular model of device.

This is why, when you plug in a new printer, the computer sometimes says "installing device driver". Without the correct driver, the peripheral is present physically but mute: the OS simply does not know how to talk to it.



Each peripheral is matched to its own driver, which acts as interpreter between the device and the operating system.

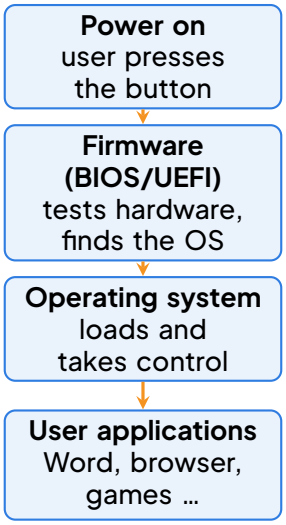
2.4 Firmware

Some software does not live on the hard disk at all: it is recorded permanently inside a hardware component itself.

DEFINITION **Firmware**

Firmware is software stored permanently in a hardware component, typically in a read-only or flash memory chip. It gives the component its basic behaviour. Examples: the BIOS/UEFI that starts the computer and tests the hardware before the OS loads, and the firmware inside a router, a printer or a digital camera.

The BIOS (Basic Input/Output System), or its modern successor UEFI, is the very first program that runs when you press the power button. It checks the memory and keyboard, then locates the operating system on the disk and hands control over to it. This start-up process is called booting.



The boot sequence: the firmware wakes the machine, then hands control to the operating system, which in turn hosts the applications.

2.5 Utility software

A house needs not only a manager but also a maintenance team: cleaners, repairers, security guards. In a computer, this team is the utility software.

DEFINITION **Utility software**

Utility software is a family of support programs that maintain, protect and optimise the computer system. They do not manage the hardware directly like the OS, but they serve the system rather than a specific user production task. Examples: antivirus programs, disk defragmenters, compression tools (WinZip, WinRAR), and backup tools.

- An **antivirus** scans files and memory to detect and remove malicious programs.
- A **disk defragmenter** rearranges scattered pieces of files on a magnetic disk so that reading is faster.
- A **compression tool** reduces the size of files to save space or speed up transfer.
- A **backup tool** copies important data to a safe location so it can be restored after a failure.

WARNING **Common mistake: the antivirus trap**

Candidates very often classify an antivirus as application software, arguing that "the user launches it like any program". In the GCE classification scheme, an antivirus is a utility, that is, system-support software: its purpose is to maintain and protect the system,

not to produce user work (a letter, an account, a drawing). The same applies to defragmenters, compression and backup tools. Learn this distinction by heart — it is a favourite exam question.

2.6 Translators and development tools

Finally, some system software is used not to run the machine but to create other software. Programs are written by humans in high-level languages (like Pascal, C or Python), but the processor understands only machine code (binary). A bridge is needed.

- An **assembler** translates assembly language (a low-level, symbolic language) into machine code.
- A **compiler** translates a whole high-level program into machine code in one pass, producing an executable file (e.g. the C compiler `gcc`).
- An **interpreter** translates and executes a high-level program line by line, each time it runs (e.g. the Python interpreter).
- A **linker** combines several compiled modules and library files into a single executable program.

These **translators** and development tools are classified as system software because they serve the general activity of computing itself, not an end-user task.

2.7 System software in the layered architecture

All these elements fit into the layered view of a computer introduced earlier: the **hardware** at the bottom; the **firmware** just above it; then the **operating system** (with its drivers) and the **utilities and translators**; and finally the **application software** used by the **user** at the top. Each layer can only use the services of the layer directly below it.

PROPERTY **The OS as indispensable intermediary**

Without system software, no application software can run. The operating system is the obligatory intermediary between the user, the applications and the hardware: an application never touches the hardware directly; it always asks the OS, which alone has the right and the knowledge to control the machine. (Statement to know; no proof is examinable.)

WORKED EXAMPLE 2 **Classifying software**

A school computer in Bamenda contains the following programs: (1) Ubuntu, (2) the BIOS, (3) an HP printer driver, (4) Avast antivirus, (5) the `gcc` compiler, (6) Microsoft Word. State

the sub-category of system software to which each of the first five programs belongs, and identify the one that is not system software.

SOLUTION

Solution.

- (1) Ubuntu: an operating system (a Linux distribution).
- (2) The BIOS: firmware — software stored permanently in a chip on the motherboard.
- (3) HP printer driver: a device driver — it lets the OS communicate with that specific printer.
- (4) Avast antivirus: a utility — system-support software that protects the system (not application software!).
- (5) The `gcc` compiler: a translator (development tool) — it transforms source code into machine code.
- (6) Microsoft Word is not system software: it is application software, since it performs a user task (word processing).

METHOD **Answering a classification question in the GCE**

1. Ask: does this program serve the machine/system, or a user production task? If it serves the system, it is system software.
2. Then refine: does it manage all resources (OS)? Link the OS to one device (driver)? Live inside a component (firmware)? Maintain or protect (utility)? Translate code (translator)?
3. Always justify your answer with one sentence and give an example when asked.

KEY POINTS **Key points**

- System software manages the hardware and provides a platform for applications.
- The OS manages the processor, memory, files, peripherals and users (examples: Windows, Ubuntu, macOS, Android, iOS).
- A driver connects the OS to one specific peripheral; firmware is software fixed inside a component (BIOS/UEFI).
- Utilities (antivirus, defragmenter, compression, backup) maintain the system; trans-

lators (assembler, compiler, interpreter, linker) help create software.

- Without system software, no application can run.

EXERCISE 2 **Know your examples**

For each sub-category of system software — operating system, device driver, firmware, utility, translator — give two examples different from those used in this lesson, and justify each choice in one sentence.

EXERCISE 3 **True or false?**

Answer true or false and justify: (a) "Android is application software because it runs on phones." (b) "A program can print a document even if no printer driver is installed." (c) "The BIOS is part of the operating system."

3 APPLICATION SOFTWARE

In the previous section we studied **system software** — the operating system, utilities, translators and drivers whose job is to manage the machine itself. None of these programs, however, writes a letter, computes a school report card or plays a song. They prepare the ground; they do not do the user's work. The programs that actually perform the tasks for which people buy computers form the second great family of software: **application software**. This section defines it, classifies it, and shows how it relates to the operating system beneath it.

3.1 Definition and intuition

Think of a restaurant in Douala. The kitchen, the electricity, the water supply and the cash register system are like system software: essential, but invisible to the customer. What the customer actually came for is the meal on the plate. Application software is the meal: it is what the user directly uses.

DEFINITION **Application software**

Application software (or simply an application or app) is software designed to perform tasks that are directly useful to the end user: producing documents, performing calculations, communicating, learning, playing or managing an activity. Unlike system software, it does not manage the computer itself; it uses the computer — through the services of the operating system — to solve a user's problem.

The test is simple: ask "does this program do the user's work, or does it serve the machine?". A word processor serves the user (application); the disk driver that saves the document serves the machine (system software).

3.2 General-purpose (off-the-shelf) application software

Many human tasks are the same everywhere: writing letters, adding up numbers, browsing the web, watching videos. For such common tasks, software companies produce standardised programs sold to millions of users.

DEFINITION **General-purpose application software**

General-purpose (or off-the-shelf) application software is ready-made software, developed by a vendor and sold or distributed to the general public, that can be used by many different users for common, everyday tasks. Examples: the word processor Microsoft Word, the spreadsheet Microsoft Excel, the browser Mozilla Firefox, the media player VLC.

EXEMPLE **A spreadsheet serves everyone**

A trader at Mokolo market uses Excel to track daily sales; a bursar in Buea uses the same Excel to prepare the school budget; a student uses it to plot a physics graph. The software was not written for any of them in particular, yet all three find it useful because calculation on tables of numbers is a general need. This is the essence of general-purpose software.

3.3 Special-purpose (bespoke) application software

Some needs are so specific that no ready-made product fits them. The software must then be tailor-made, like a dress sewn by a tailor to one customer's measurements.

DEFINITION **Special-purpose (bespoke) software**

Special-purpose (also called bespoke or customised) application software is software developed for one specific organisation or one specific task, according to that organisation's exact requirements. It is not sold to the general public.

EXEMPLE **Bespoke software in Cameroon**

- A school fees management system written for a college in Yaoundé, matching its exact classes, fee instalments and receipt format.

- The examination processing system used to process GCE scripts and results.
- A bank's core banking software managing its accounts and branches.
- The MTN Mobile Money platform, built specifically for MTN's mobile money service.

None of these can be bought in a shop; each was built for one organisation's precise needs.

3.4 General-purpose versus bespoke software

General-purpose (off-the-shelf)	Special-purpose (bespoke)
Ready-made, sold to many users for common tasks	Built to order for one organisation or one task
Examples: MS Word, Excel, Firefox, VLC	**Examples:** fees system of a college, MTN Mobile Money
Cost: low per user (shared by millions)	**Cost:** high (one client pays the whole development)
Advantages: cheap, available immediately, well tested, regular updates, wide support	**Advantages:** fits the needs exactly, no useless features, competitive advantage
Limitations: may not match every need; user must adapt to the software	**Limitations:** expensive, long to develop, maintenance depends on the developer

Comparison of general-purpose and special-purpose application software.

METHOD **Deciding between off-the-shelf and bespoke software**

When an organisation must choose, proceed as follows:

1. Analyse the need. List exactly what the software must do. If the need is common (letters, accounts, presentations), an off-the-shelf product probably exists.
2. Compare costs. Off-the-shelf software is cheap per user; bespoke software requires paying analysts and programmers for months.
3. Consider time. Off-the-shelf software can be installed today; bespoke software

may take months or years to design, code and test.

4. Examine maintenance and support. Who will fix bugs and adapt the software in five years? A vendor offers standard support; bespoke software ties you to its developer.
5. Decide. Choose bespoke only when no existing product fits the need and the organisation can afford the cost and delay.

WARNING **Bespoke is not always better**

A frequent mistake is to believe that custom-made software is automatically superior. Bespoke software is costly (one client bears the whole development cost), slow to obtain (analysis, coding, testing), and risky to maintain (if the developer disappears, nobody else knows the code). For common tasks, a well-tested off-the-shelf product is usually the wiser choice.

3.5 Categories of applications by domain

Application software can also be classified by the **domain of activity** it serves:

Domain	Purpose	Examples
Office automation	Produce documents, calculations, presentations	Word, Excel, PowerPoint
Graphics	Create and edit images, drawings, designs	Photoshop, CorelDRAW, GIMP
Multimedia	Play and edit sound and video	VLC, Windows Media Player
Communication	Exchange messages and browse the web	Firefox, WhatsApp, Outlook
Education	Teach, train and assess learners	Typing tutors, encyclopaedias, e-learning platforms
Games	Entertainment	FIFA, chess programs
Scientific software	Simulation, modelling, data analysis	MATLAB, statistical packages

3.6 Single applications, integrated packages and software suites

Applications can be distributed in two ways:

DEFINITION **Single application, integrated package and suite**

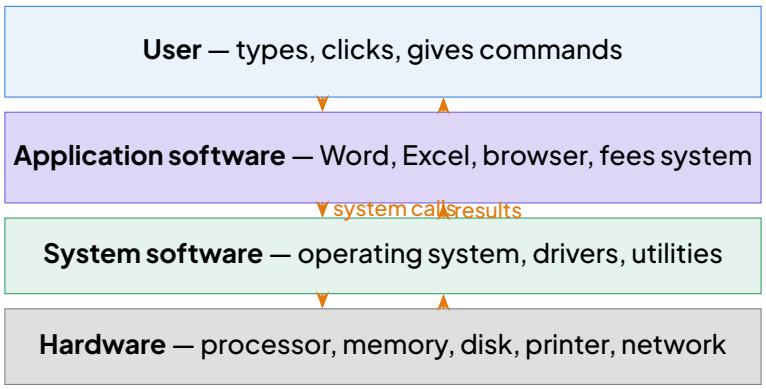
A **single application** is one program sold on its own to do one type of task (e.g. VLC).

An **integrated package** is a single program combining several related functions (e.g. Microsoft Works combined a simple word processor, spreadsheet and database in one). A **software suite** is a collection of separate but compatible applications sold together and sharing data easily, such as Microsoft Office (Word, Excel, PowerPoint, Access) or the free suite LibreOffice (Writer, Calc, Impress, Base).

The great advantage of a suite is **consistency and integration**: the same menus and shortcuts everywhere, and the possibility to paste an Excel chart into a Word report without losing its link to the data.

3.7 How applications relate to the operating system

Applications do not talk to the hardware directly. When Word saves a file or a browser sends data over the network, the application **calls a service of the operating system** (a system call), and the OS performs the operation through its drivers.



The software layers revisited: applications sit on top of the operating system and request its services.

KEY POINTS Key points

- Application software performs tasks directly useful to the user.
- General-purpose software is ready-made for common tasks (Word, Excel); bespoke software is built for one organisation's exact needs (fees system, Mobile Money platform).
- Bespoke software fits perfectly but is expensive and slow to develop.
- Applications are classified by domain: office, graphics, multimedia, communication, education, games, scientific.

- Suites (MS Office, LibreOffice) bundle compatible applications; applications run on top of the OS and call its services.

WORKED EXAMPLE 3 **Classifying software at the GCE**

Classify each of the following as system software, general-purpose application software or special-purpose application software, justifying your answer: (a) Microsoft Excel; (b) the result-processing software of the GCE Board; (c) Windows 11; (d) VLC media player.

SOLUTION

Solution.

- (a) General-purpose application software: Excel performs calculations directly useful to any user, and it is a ready-made product sold to the public for the common task of working on tables of numbers.
- (b) Special-purpose application software: it serves users (processing examination results) but was developed specifically for the GCE Board's own procedures; it is not sold to the public.
- (c) System software: Windows 11 manages the hardware and provides services to other programs; it does not itself produce a user document.
- (d) General-purpose application software: VLC plays audio and video — a common task needed by millions of users — and is distributed ready-made to everyone.

WARNING **Exam tip**

When a question asks you to classify a software, always give one concrete example and justify your classification with the software's purpose (what task it does and for whom it was made). A bare answer such as "application software" without justification earns little credit.

EXERCISE 4 **Choosing software for a school**

A new secondary school in Bamenda wants to (i) type official letters, and (ii) manage students' fees with instalments specific to its own rules. For each need, say whether you would recommend off-the-shelf or bespoke software, and justify using cost, fit to needs and development time.

SOLUTION **Exercise 1**

(i) Typing letters is a common task: an off-the-shelf word processor (e.g. Word or LibreOffice Writer) is cheap, available immediately and fully sufficient. (ii) The fees rules are specific to the school; if no existing product matches them, bespoke software is justified despite its higher cost and longer development time, because it will fit the need exactly. A cheaper alternative is to adopt an existing school-management package if one can be configured to the school's rules.

4 CLASSIFICATION BY USAGE RIGHTS AND SYNTHESIS

In the previous sections we classified software according to what it does: system software keeps the machine running, while application software performs tasks for the user. But there is another question that matters enormously in practice, especially for a school, a business or a government office in Cameroon that must equip its computers: **what are you legally allowed to do with the software, and what does it cost?** Two programs can do exactly the same job — think of Microsoft Office and LibreOffice — yet be governed by completely different rules. This section studies the classification of software **by usage rights** (also called classification **by licence**), and then brings together all the classification criteria seen in this chapter into one complete synthesis method.

4.1 The software licence

EXPERIMENT **Buying a CD vs. buying software**

When you buy a music CD in a shop in Douala, the disc belongs to you, but you are not allowed to copy it and sell the copies: you bought the object, not the rights over the work. Software works the same way, only more strictly. When you "buy" software, you usually do not buy the program itself; you buy the right to use it under precise conditions. Those conditions are written in a contract that you accept (often by clicking "I agree") during installation.

DEFINITION **Software licence**

A **software licence** is the legal contract between the author (or owner) of a software and the user. It defines what the user may and may not do with the software: on how many machines it may be installed, whether it may be copied, modified, resold or redistributed, and under what payment conditions.

The licence is what makes the classification by usage rights possible. Depending on what the

licence allows, software falls into one of five great families: **proprietary software**, **freeware**, **shareware**, **open source software** and **public domain software**. The key questions to ask are always the same three:

- **Must I pay** to use it?
- **Can I see the source code** (the original program text written by the programmers)?
- **Can I modify and redistribute** it?

4.2 Proprietary software

DEFINITION **Proprietary software**

Proprietary software is software whose source code is closed (kept secret by its owner) and whose use is restricted by a paid licence. The user pays for the right to use the program, but may not study, modify, copy or redistribute it.

This is the traditional commercial model. The company invests money in development and protects its product like any other property. Well-known examples are **Microsoft Windows** (operating system), **Microsoft Office** (Word, Excel, PowerPoint) and **Adobe Photoshop** (image editing). A school in Yaoundé that installs Windows on 30 computers must, in principle, hold 30 valid licences.

4.3 Freeware

DEFINITION **Freeware**

Freeware is software that is free of charge to use (gratis), but whose source code remains closed and whose copyright is retained by the author. The user may use it without paying, but may not study, modify or (usually) redistribute it freely.

Typical examples: **Adobe Acrobat Reader** (reading PDF files) and **Skype** (Internet calls). You pay nothing, but the program remains the private property of its publisher.

PROPERTY **"Free of charge" does not mean "open source"**

Gratuity and openness are two independent properties. Freeware proves it: it costs nothing, yet its source code is closed and its copyright fully retained. Conversely, some open source software is sold commercially (e.g. Red Hat Enterprise Linux). Free of charge speaks about price; open source speaks about access to the code and freedom to modify it.

4.4 Shareware

DEFINITION **Shareware**

Shareware is software distributed free of charge for a limited trial period (or with limited functionality), after which payment is required to continue using it legally or to unlock all features.

The idea is "try before you buy". Classic examples: **WinRAR** (file compression, which keeps working after the trial but legally requires payment) and the **trial versions of antivirus** programs (e.g. 30 days free, then a subscription). Shareware is a marketing model built on top of proprietary software: the code stays closed, and after the trial it behaves like ordinary proprietary software.

4.5 Open source software

DEFINITION **Open source software**

Open source software is software whose source code is available to everyone and whose licence grants four freedoms: to use the program, to study it, to modify it, and to redistribute it (modified or not).

Famous examples: **Linux** (operating system, e.g. the Ubuntu distribution), **LibreOffice** (office suite), **Mozilla Firefox** (web browser) and **VLC** (media player, developed originally by French students and used worldwide, including in practically every Cameroonian computer room). Open source software is usually also free of charge, but its defining feature is the openness of the code, not the price.

DID YOU KNOW? **Free software vs. open source**

For GCE essay questions, it is enriching to know the nuance. The free software movement (launched by Richard Stallman in the 1980s) defends software freedom as an ethical principle: users deserve freedom over their tools. The term open source (popularised in 1998) emphasises the practical advantages of open code: better quality, security and collaboration. In practice the two labels cover almost the same licences (GNU GPL, Apache, MIT…); the difference is one of philosophy and vocabulary, not of software lists. In Cameroon, open source software is precious for schools and administrations because it removes licence costs and allows local adaptation.

4.6 Public domain software

DEFINITION **Public domain software**

Public domain software is software with no copyright at all: the author has explicitly abandoned all rights (or the rights have expired). Anyone may use, copy, modify, redistribute and even sell it, without any restriction.

This is the freest possible status — freer even than open source, because open source licences still impose conditions (for example, the GPL requires that redistributed versions remain open). Public domain software is comparatively rare; it is found mainly among old programs, academic code explicitly released without rights, and small utilities.

WARNING **Confusions that cost marks**

- Freeware ≠ open source. Freeware is gratis but closed; open source may even be sold but its code is open. Acrobat Reader is freeware, not open source.
- Shareware ≠ freeware. Shareware is free only temporarily (trial); freeware is free permanently. WinRAR is shareware, not freeware.
- Open source ≠ public domain. Open source still has a copyright holder and a licence with conditions; public domain has no rights at all.
- "Free" is ambiguous in English. Always ask: free as in price (gratis) or free as in freedom (libre)?



The spectrum of usage rights, from fully proprietary software to the public domain, with five familiar examples.

4.7 Synthesis: classifying a software completely

We have now met three independent classification criteria in this chapter:

1. **By role**: system software (operating system, utilities, drivers, translators) vs. application software.
2. **By purpose** (for application software): general-purpose (word processor, browser…) vs. special-purpose / bespoke (school-fee management, hospital records…).

3. **By licence** (usage rights): proprietary, freeware, shareware, open source, public domain.

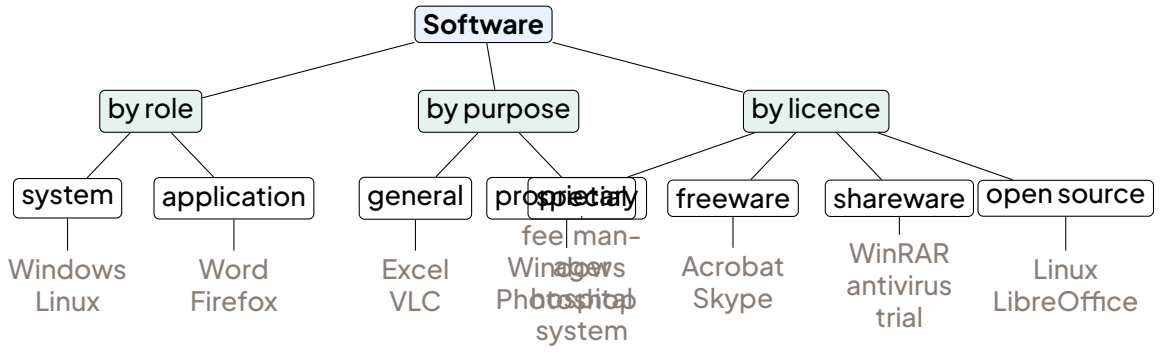
A complete classification of a software product answers all three questions.

METHOD **The 3-step classification of any software**

1. Role. Ask: does it manage the machine or serve the user's task? → system or application software. Justify by stating what it does.
2. Purpose. If it is application software, ask: is it used by everyone for common tasks, or built for one specific activity? → general-purpose or special-purpose.
3. Licence. Ask: must one pay? is the source code open? may it be modified and redistributed? → proprietary, freeware, shareware, open source or public domain. Justify with the licence facts.

EXEMPLE **Complete classification of VLC**

VLC media player is: application software (it serves the user's task: playing audio and video), general-purpose (used by anyone, for a common task), open source (its code is published and modifiable) and free of charge. Four precise statements — that is what a full GCE answer looks like.



Summary tree of the complete classification of software, with examples hanging from each leaf (public domain, the fifth licence category, has no famous everyday example).

WORKED EXAMPLE 4 **Classifying software in a scenario**

Statement. A cybercafé in Bamenda uses the following software: (a) Microsoft Windows 10, paid per machine; (b) WinRAR, past its 40-day trial; (c) Mozilla Firefox; (d) Adobe Ac-

robat Reader. For each one, give its complete classification (role, purpose, licence), justifying briefly.

SOLUTION

Solution.

- Windows 10: system software (it manages the hardware and runs other programs — it is an operating system); purpose criterion does not apply to system software; proprietary (paid licence, closed source code).
- WinRAR: application software (compresses files for the user), general-purpose (common task for everyone), shareware (free trial, payment required afterwards — using it past the trial without paying is not legal use).
- Mozilla Firefox: application software, general-purpose (web browsing), open source (code published, modifiable, redistributable) and free of charge.
- Acrobat Reader: application software, general-purpose (reading PDF documents), freeware (gratis, but closed code and copyright retained by Adobe).

4.8 Revision grid and GCE question patterns

KEY POINTS **Revision grid**

Category	Pay?	Source code?	Examples
Proprietary	Yes (licence)	Closed	Windows, MS Office, Photoshop
Freeware	No	Closed	Acrobat Reader, Skype
Shareware	After trial	Closed	WinRAR, antivirus trials
Open source	Usually no	Open (4 freedoms)	Linux, LibreOffice, Firefox, VLC
Public domain	No	No copyright at all	rare old/academic programs

METHOD **Answering GCE scenario questions**

GCE questions frequently describe a school or a business and ask you to propose and jus-

tify software. For full marks, always give three elements per proposal:

1. Name the category (e.g. “an open source office suite”);
2. Give a concrete example (e.g. “LibreOffice”);
3. Justify with the scenario’s needs (e.g. “the school has no budget for licences, and open source software is free of charge and legally installable on all machines”).

A category without an example, or an example without justification, loses marks.

EXERCISE 5 **Practice**

1. A hospital in Buea pays a programmer to write a patient-records program used only by that hospital, and keeps the code secret. Classify this software by role, purpose and licence.
2. True or false (justify): “Skype is free, therefore it is open source.”
3. Explain the difference between shareware and freeware, giving one example of each.
4. Your school wants to equip 40 computers with an office suite but has almost no budget. Propose a solution, naming the category, an example, and two justifications.

SOLUTION **Exercise 1**

1. Role: application software (it serves the users’ task, not the machine). Purpose: special-purpose (bespoke), built for one specific activity of one specific hospital. Licence: proprietary (closed code, owned by the hospital/developer, not redistributable).
2. False. Skype is free of charge (freeware), but its source code is closed and its copyright belongs to its owner. Gratuity does not imply open source.
3. Freeware is free permanently (e.g. Acrobat Reader); shareware is free only for a trial period, after which payment is required (e.g. WinRAR). Both have closed source code.
4. Category: open source office suite; example: LibreOffice. Justifications: it is free of charge, so the school pays nothing for 40 installations; it is legally installable on all machines without counting licences; and its files remain compatible with common formats, so work can be exchanged with outside partners.

5 INTEGRATION ACTIVITY

5.1 Aims of the activity

This activity mobilises the classification criteria to select and justify a complete software set for a real school situation.

5.2 Situation

The principal of GBHS Bamenda has just received 20 new computers for the computer laboratory but no software is installed. In groups, students must: (1) list the software needed (operating system, office suite, antivirus, browser, a typing tutor, and a school management program for results); (2) classify each chosen software by role, by purpose and by licence; (3) propose two budgets — one fully proprietary and one maximising free/open source software — and compare costs; (4) defend their choices in a 5-minute presentation, as in a GCE structured question.

5.3 Tasks

Guided numbered questions mobilising several skills progressively.

5.4 Model answer

A complete worked answer.

5.5 ...

6 METHODS AND WORKED EXAMPLES

6.1 Methods and worked examples

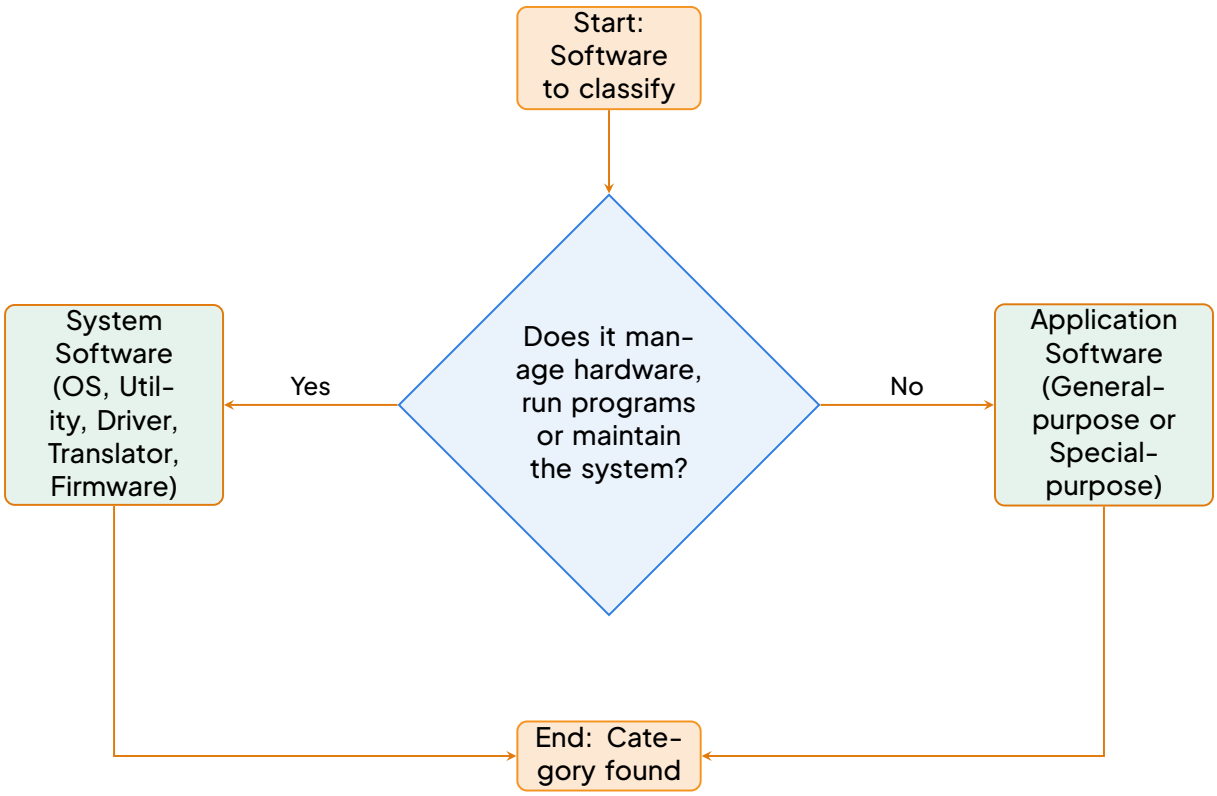
This part trains you, step by step, on the skills the GCE Advanced Level examiner expects for this chapter: recognising the category of a given software, justifying a classification, distinguishing system software from application software, classifying by usage rights (freeware, shareware, proprietary, open source), and building complete classification tables or trees. Each method card is followed by a fully solved GCE-style exercise.

METHOD **Classifying a software into the two great families**

To decide whether a given software is **system software** or **application software**, proceed as follows:

1. Ask the key question: does the software serve the machine (manage, control, maintain the computer and its resources) or does it serve the user's task (write a letter, compute a payroll, play a game)?
2. If it manages hardware, runs other programs, or maintains the system ⇒ system software.
3. If it solves a concrete user problem in a specific domain ⇒ application software.
4. Refine the classification: for system software, say whether it is an operating system, a utility, a driver, a translator (compiler/interpreter) or firmware; for application software, say whether it is general-purpose (word processor, spreadsheet) or special-purpose (payroll, hospital management).
5. Justify your answer in one sentence: name the resource managed or the task performed. A classification without justification earns only half the marks.

Reflex: "Who benefits directly — the machine or the user's work?"



Flowchart for the fundamental classification: system vs application software.

WORKED EXAMPLE 5 **Sorting a list of software**

(GCE style, Paper 2) The following software are installed on the computers of a secondary school in Bamenda: Windows 10, Microsoft Word, Avast Antivirus, Google Chrome, a printer driver, Microsoft Excel, a C++ compiler, Adobe Photoshop.

1. Copy and complete a table classifying each software as system software or application software.
2. For each system software, state its precise sub-category.

SOLUTION

Solution.

1. We apply the key question to each item: does it serve the machine or the user's task?

Software	Category	Justification
Windows 10	System software	Manages hardware resources and runs other programs
Microsoft Word	Application software	Performs the user task of word processing
Avast Antivirus	System software	Maintains and protects the system (utility)
Google Chrome	Application software	Performs the user task of browsing the web
Printer driver	System software	Enables the OS to control a hardware device
Microsoft Excel	Application software	Performs the user task of calculations/spreadsheets
C++ compiler	System software	Translates source programs into machine code
Adobe Photoshop	Application software	Performs the user task of image editing

2. Sub-categories of the system software:
   - Windows 10: operating system;
   - Avast Antivirus: utility program;
   - Printer driver: device driver;
   - C++ compiler: translator (language processor).

Examiner's remark: an antivirus is a classic trap — students often call it "application software" because the user clicks on it. It is a utility, hence system software, because its role is to maintain and protect the computer itself.

KEY POINTS **System vs Application — the litmus test**

- System software = "backstage crew": the user rarely interacts with it directly; it creates the environment where applications run.
- Application software = "actors on stage": the user launches it to achieve a concrete goal (document, calculation, game, browsing).
- Utility programs (antivirus, defragmenter, backup) are system software because they maintain the computer, not the user's document.

METHOD **Distinguishing general-purpose from special-purpose application software**

1. Identify the domain of use of the software.
2. If the software can be used in many different domains by many kinds of users (typing text, calculating, drawing, browsing) ⇒ general-purpose (also called off-the-shelf or productivity software).
3. If the software was built for one specific task in one specific organisation or profession (payroll of a company, students' results management in a school, flight booking) ⇒ special-purpose (also called bespoke or customised software).
4. Give one example of each type to support your answer.

Key idea to remember: general-purpose = one product, many possible uses; special-purpose = one product, one precise use.

WORKED EXAMPLE 6 **General-purpose or special-purpose?**

(GCE style) A cybercafé in Douala uses: (i) Microsoft Excel to record daily income; (ii) a program written by a local developer specifically to manage customers' browsing time and print their bills; (iii) VLC media player.

1. Classify each software as general-purpose or special-purpose application software.
2. State one advantage and one disadvantage of special-purpose software for the cybercafé owner.

SOLUTION

Solution.

1. Classification:
   - Microsoft Excel: general-purpose. It is a spreadsheet used in countless domains (accounting, statistics, timetables…); the cybercafé only adapted it to its own records.
   - Browsing-time management program: special-purpose. It was designed for one precise task (managing sessions and billing) for this particular business; it cannot be used, for example, to manage a pharmacy.
   - VLC media player: general-purpose. Any user can play audio and video files with it, whatever the context.
2. Advantage: the software fits the owner's exact needs (it counts browsing minutes and prints bills in the local format), so work is faster and no unnecessary features disturb the user. Disadvantage: it is expensive and slow to obtain, because a developer

must be paid to design, write and maintain it; if the developer disappears, maintenance becomes difficult.

Examiner's remark: notice that the same Excel could be part of a special-purpose solution if a developer built macros around it — what matters is the design intention: built for everyone or built for one specific need.

METHOD **Identifying the sub-category of system software**

When asked to name the type of a system software, test the following questions in order:

1. Does it manage the whole machine (memory, processor, files, users, peripherals) and provide the platform for other programs? ⇒ Operating system (Windows, Linux, Android, macOS).
2. Does it let the OS communicate with one hardware device? ⇒ Device driver.
3. Does it translate programs written by humans into machine language? ⇒ Translator: compiler (translates the whole program at once) or interpreter (translates and executes line by line).
4. Does it maintain, protect or optimise the computer (antivirus, disk defragmenter, backup, compression)? ⇒ Utility.
5. Is it stored permanently on a chip inside a device and starts or controls it? ⇒ Firmware (e.g. the BIOS/UEFI on a motherboard, the firmware of a router or printer).

Reflex: "Manage → OS; connect a device → driver; translate → compiler/interpreter; maintain → utility; on a chip → firmware."

WORKED EXAMPLE 7 **Naming sub-categories**

(GCE style) For each of the following, give the precise sub-category of system software and justify briefly:

a) The BIOS stored on the motherboard, which starts the computer.

b) A disk defragmenter delivered with Windows.

c) The Python interpreter used in the computer science laboratory.

d) The small program installed so that the new HP scanner works.

e) Ubuntu Linux running on the school server.

SOLUTION

Solution.

a) Firmware. It is software permanently recorded on a ROM/flash chip of the motherboard; it initialises and tests the hardware at start-up, then loads the operating system.

b) Utility. It maintains the disk by reorganising file fragments so that access is faster; it neither manages the whole machine nor performs a user task like writing.

c) Translator (interpreter). It converts each line of a Python source program into machine-executable form and executes it immediately, line by line.

d) Device driver. It acts as the interface between the operating system and that precise scanner model; without it the OS cannot send correct commands to the device.

e) Operating system. It manages the processor, memory, files, users and peripherals of the server and provides the environment in which all other programs run.

Examiner's remark: always give the precise word. Writing "system software" when the question asks for the sub-category is incomplete and loses marks.

METHOD **Classifying software by usage rights (licences)**

To classify a software according to its **licence** (usage rights), ask three questions:

1. Must one pay to use it legally?
   - Yes, and the source code is secret ⇒ proprietary (commercial) software (e.g. Microsoft Office, Windows, Adobe Photoshop).
   - No payment at all ⇒ go to question 2.
   - Free for a trial period or with limited functions, payment afterwards ⇒ shareware (e.g. WinRAR, some games with "demo" versions).
2. Is the source code available and modifiable, with the right to redistribute?
   - Yes ⇒ open source / free software (e.g. Linux, LibreOffice, GIMP, Firefox, Python).
   - No, only the executable is given free of charge ⇒ freeware (e.g. Adobe Acrobat Reader, Skype, TeamViewer free version).
3. Is it free of any copyright restriction? ⇒ public domain software (rare, e.g. some very old programs or explicit donations).

Key ideas: freeware = free of charge but closed; open source = free and modifiable; shareware = "try before you buy"; proprietary = pay and no access to the code.

WORKED EXAMPLE 8 **Licences in real situations**

(GCE style) For each situation, name the type of licence concerned and justify:

a) A school downloads LibreOffice; it may study its source code, adapt it and copy it freely for all its computers.

b) A student uses WinRAR free of charge; after 40 days a message asks him to buy a licence, though the program keeps working.

c) A company buys Microsoft Office 365 and receives only the installed programs, never the source code.

d) A teacher installs Adobe Acrobat Reader free of charge to read PDF files, but cannot modify the program.

SOLUTION

Solution.

a) Open source (free) software. The four freedoms are present: use, study (source code available), modify, and redistribute. LibreOffice is a standard example.

b) Shareware. The software is distributed free for an evaluation period (here 40 days); continued legal use requires payment. “Try before you buy” is the signature of shareware.

c) Proprietary (commercial) software. The company pays for a licence of use only; the source code remains the secret property of Microsoft and modification or redistribution is forbidden.

d) Freeware. The executable is given at no cost, but the source code is closed and the software cannot be modified — free of charge does not mean free to modify.

Examiner’s remark: the most frequent confusion is between freeware and open source. Remember: in freeware only the price is zero; in open source the freedom (source code, modification, redistribution) is the essential point.

KEY POINTS **Licence types at a glance**

Type	Cost	Source code	Typical examples
Proprietary	Paid	Closed	Windows, MS Office, Photoshop
Shareware	Free trial, then paid	Closed	WinRAR, some utilities
Freeware	Free	Closed	Acrobat Reader, Skype
Open Source	Free	Open (modifiable)	Linux, LibreOffice, Python, Firefox
Public Domain	Free	Open (no copyright)	Rare, very old code

METHOD **Comparing compiler and interpreter**

When a question opposes these two translators, structure the answer as a comparison on precise criteria:

1. Moment of translation: compiler = whole source program translated once, producing an executable file; interpreter = translation line by line, at each execution.
2. Speed of execution: compiled code runs faster (already in machine language); interpreted code is slower.
3. Error handling: the compiler lists all errors after scanning the whole program; the interpreter stops at the first error, which eases debugging for beginners.
4. Memory / portability: the interpreter must be present on every machine that runs the program; the compiled executable runs alone but only on the platform for which it was compiled.
5. Examples: compiler → C, C++; interpreter → Python, BASIC (classic).

Reflex: present the comparison in a two-column table — examiners reward the clear layout.

WORKED EXAMPLE 9 **Compiler versus interpreter**

(GCE style) The computer science teacher of GBHS Yaoundé explains that the C++ programs of Upper Sixth are compiled, while the Python scripts of Lower Sixth are interpreted.

1. Construct a table giving four differences between a compiler and an interpreter.
2. Explain why the interpreter is often preferred when learning programming.

SOLUTION

Solution.

1. Comparison table:

Criterion	Compiler	Interpreter
Translation	Whole program translated at once into an executable file	Line by line, at every execution
Execution speed	Fast (machine code ready)	Slower (translation each time)
Errors	All errors listed after full analysis	Stops at the first error encountered
Presence needed	Executable runs without the compiler	Interpreter must be installed on each machine

2. When learning, the beginner makes many mistakes. The interpreter executes the program immediately and stops exactly at the first faulty line, displaying a message at once: the learner can correct and re-run within seconds, without a separate compilation step. This rapid "write–test–correct" cycle makes experimentation easy, which is why Python (interpreted) is chosen for Lower Sixth, while compiled languages are appreciated later for producing fast, stand-alone executables.

METHOD **Building a complete classification tree or table**

For "classify the following software" questions worth many marks:

1. Draw the two main branches first: System software and Application software.
2. Subdivide: system software → operating systems, utilities, drivers, translators, firmware; application software → general-purpose and special-purpose.
3. Place each given software at the deepest possible level (the most precise branch).
4. Add the licence dimension if requested: proprietary, freeware, shareware, open source — note that this criterion is independent of the system/application criterion (an OS can be proprietary like Windows or open source like Linux).
5. Check: every software of the list appears exactly once, and each placement is justified by the function of the software.

Warning: never mix the two classification criteria on the same branch level (e.g. do not put "open source" as a sub-branch of "system software").

WORKED EXAMPLE 10 **Full classification of a computer's content**

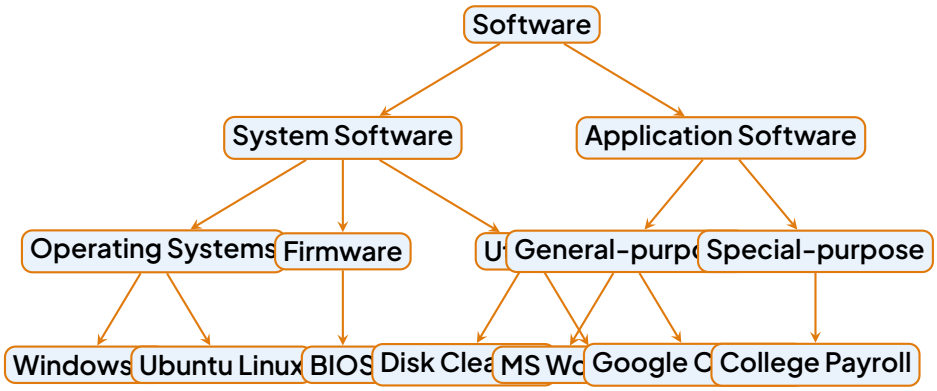
(GCE style, structured question) A new laptop offered to the best student of a college in Buea contains: Windows 11 (paid licence), the BIOS, Microsoft Word (paid), a disk cleanup tool, Linux Ubuntu (installed in dual boot, source code available), a payroll program developed for the college bursar, Google Chrome (free, closed code), and an antivirus.

1. Draw a classification tree placing each software in its correct family and subcategory.
2. Indicate, for each software, its type of licence where the information allows it.

SOLUTION

Solution.

1. Classification tree (function criterion):



Functional classification tree for the laptop software.

2. Licences (usage-rights criterion, independent of the tree above):

Software	Licence
Windows 11	Proprietary (paid, closed code)
Microsoft Word	Proprietary (paid)
Ubuntu Linux	Open source (free, source code available)
Google Chrome	Freeware (free of charge, closed code)
BIOS, cleanup tool, antivirus, payroll	Not stated in the text — cannot be determined

Examiner's remark: note the honesty of the last table row: a classification must rest on given information. Also observe that Windows and Ubuntu sit in the same functional box (operating systems) although their licences differ — proof that the two criteria are inde-

pendent.

METHOD **Arguing the choice of a software for a given context**

For open questions of the type “advise this company / this school”:

1. Analyse the needs: tasks to perform, budget, number of machines, skills of users, need for maintenance.
2. Match needs to categories: which OS, which utilities, which general-purpose applications, is a special-purpose development justified?
3. Discuss the licence: open source reduces cost and allows adaptation but may require more technical skill; proprietary software offers commercial support but costs licences per machine.
4. Conclude with a justified recommendation (at least two arguments). In the GCE, a recommendation without justification scores very little.

WORKED EXAMPLE 11 **Advising a new computer laboratory**

(GCE style, essay-type) A newly created Government High School in the Far North Region receives 20 computers but has a very small budget for software. The principal asks your advice for equipping the machines so that students can learn word processing, spreadsheets and programming.

1. Propose a complete software equipment (system and application software), naming the category of each element.
2. Justify your choices with regard to the budget constraint.

SOLUTION

Solution.

1. Proposed equipment:
   - Operating system (system software): Ubuntu Linux — open source, so no licence to pay for the 20 machines.
   - Office suite (general-purpose application software): LibreOffice (Writer for word processing, Calc for spreadsheets) — open source and free, with functions equivalent to commercial suites.
   - Programming environment (translator, system software): the Python interpreter — free and open source, ideal for learning.

- Utility (system software): a free antivirus or the built-in security tools of Ubuntu, plus a backup utility.
- Web browser (general-purpose application software): Firefox or Chromium — open source.

2. Justification: with 20 machines, proprietary licences (e.g. an operating system plus an office suite per machine) would multiply the cost twentyfold and exceed the small budget. Open source software costs nothing in licences, can legally be installed on all 20 computers and copied for students, and its source code can even serve as study material in computer science lessons. The functions needed (typing documents, calculations, learning programming) are fully covered. The only trade-off is the need for a teacher or technician able to install and maintain Linux, which is a training investment rather than a financial one.

Examiner's remark: this answer scores highly because every proposed element is placed in a category of the chapter and every choice is justified by the context — exactly what the marking guide rewards.

WARNING **Frequent traps in classification questions**

- Calling an antivirus application software: it is a utility, hence system software.
- Confusing freeware (free of charge, closed) with open source (source code available and modifiable).
- Believing that "system software" is an acceptable answer when the sub-category (OS, driver, utility, translator, firmware) is requested.
- Mixing the two criteria (function and licence) in the same classification tree.
- Forgetting that one software can be described on both axes: e.g. Linux is an operating system and open source.
- Classifying without a one-line justification: at the GCE, the justification carries marks.

7 EXERCISES

8 CATEGORIES OF SOFTWARE

8.1 Software: concept and classification criteria

DEFINITION **Software**

Software is the set of programs, procedures, rules, documentation and associated data that enable a computer system to perform specific tasks. Unlike hardware (the physical components), software is intangible and stored on storage media.

PROPERTY **Classification criteria**

Software can be classified according to three independent criteria:

1. Role (function): what the software does in the computer system — system software vs application software.
2. Purpose (specific function): the precise task it performs — operating system, utility, device driver, firmware, general-purpose application, bespoke application, etc.
3. Usage rights (licence): the legal conditions under which it may be used, copied, modified and distributed — proprietary, freeware, shareware, open source.

KEY POINTS **Independence of classifications**

The three classifications are independent: a single piece of software belongs to one category in each classification simultaneously. For example, Ubuntu is (role) system software, (purpose) an operating system, and (licence) open source. Windows 11 is (role) system software, (purpose) an operating system, and (licence) proprietary.

8.2 System software

DEFINITION **System software**

System software is a collection of programs designed to operate, control, and extend the processing capabilities of the computer hardware itself. It provides the platform on which application software runs and manages hardware resources (CPU, memory, storage, I/O devices).

PROPERTY **Main sub-categories of system software**

1. Operating system (OS): the master program that manages all hardware and software resources, provides a user interface, and runs other programs. Examples: Windows 11, Ubuntu Linux, macOS, Android, iOS.
2. Device drivers: programs that allow the operating system to communicate with a specific hardware device (printer, graphics card, keyboard, etc.). Each device requires its own driver.
3. Utility software (utilities): programs that perform maintenance, optimisation, or security tasks on the system. Examples: disk defragmenter, antivirus, backup tool, file compressor (WinRAR, 7-Zip), disk cleaner.
4. Firmware: low-level software permanently stored in read-only memory (ROM, EEPROM, flash) on a hardware device. It controls the device's basic operation. Examples: BIOS/UEFI on a motherboard, firmware of a router, printer, or SSD controller.
5. Language translators (system tools): compilers, interpreters, assemblers, linkers, loaders — used to develop other software but considered system software because they are essential for the software development environment.

EXEMPLE **BIOS / UEFI**

The BIOS (Basic Input/Output System) or its modern replacement UEFI (Unified Extensible Firmware Interface) is firmware stored on a chip on the motherboard. It runs immediately when the computer is powered on, performs the Power-On Self-Test (POST), initialises hardware, and loads the operating system bootloader from the storage device.

WARNING **Utility vs application**

A utility program (e.g. an antivirus or a file compressor) is system software because it acts on the system infrastructure (disks, files, security) and is often used by the OS itself. A general-purpose application (e.g. a word processor or a media player) is application software because it serves a user task unrelated to system management.

8.3 Application software

DEFINITION **Application software**

Application software consists of programs designed to help users perform specific tasks unrelated to the management of the computer system itself: writing documents, calculating, drawing, playing media, managing accounts, etc.

PROPERTY **Main sub-categories of application software**

1. General-purpose applications (off-the-shelf): mass-produced programs sold or distributed to a wide audience for common tasks. Examples: Microsoft Word, Excel, VLC media player, Adobe Photoshop, web browsers.
2. Bespoke (custom-made) applications: software developed specifically for one organisation according to its unique requirements. Examples: a payroll program for a specific company, a hospital patient management system tailored to that hospital's workflow.
3. Specialised / vertical-market applications: off-the-shelf packages targeting a specific industry (e.g. CAD for architects, accounting packages for businesses, library management systems).

KEY POINTS **Off-the-shelf vs bespoke**

- Off-the-shelf: cheaper, immediately available, tested by many users, but may not fit exact needs; support and updates depend on vendor.
- Bespoke: fits exact requirements, competitive advantage, full control over features and data, but expensive, time-consuming to develop, requires in-house or contracted expertise, maintenance burden.

8.4 Classification by usage rights and synthesis

DEFINITION **Proprietary software**

Software whose source code is not made available to the public. The licence restricts copying, modification, and redistribution. The user buys a licence to use, not ownership. Examples: Windows 11, Microsoft Office, Adobe Photoshop, WinRAR (shareware model).

DEFINITION **Freeware**

Software distributed at no cost for unlimited use, but the source code is not provided and modification/redistribution is usually forbidden. The copyright holder retains all rights. Examples: Adobe Acrobat Reader, VLC media player (note: VLC is actually open source; a better freeware example is Skype (free version) or some proprietary drivers).

DEFINITION **Shareware**

Software distributed free for a trial period or with limited functionality. After the trial, the user must pay for a licence key to continue using it or unlock full features. Examples: WinRAR (trialware), many commercial utilities.

DEFINITION **Open source software**

Software whose licence grants users the freedom to run, study, modify, and redistribute the software and its source code. The Open Source Initiative (OSI) defines precise criteria. Examples: Ubuntu Linux, LibreOffice, VLC media player, Firefox, GIMP, Python, MySQL.

PROPERTY **Common open source licences**

- GPL (GNU General Public License): copyleft — derivative works must be distributed under the same licence.
- MIT / BSD / Apache: permissive — allow proprietary derivatives, only require attribution.

WARNING **Freeware ≠ Open source**

Freeware is free of charge but closed source. Open source is free as in freedom (may be free of charge or sold). Do not confuse the two.

EXPERIMENT **Classifying a software product**

Take any software you know (e.g. `Google Chrome`, `LibreOffice Writer`, `Android OS`, a

`printer driver`). Classify it by the three criteria:

1. Role: system or application?
2. Purpose: OS, utility, driver, firmware, general-purpose app, bespoke app?
3. Licence: proprietary, freeware, shareware, open source?

Justify each classification in one sentence.

DID YOU KNOW? **Cameroon context**

The Government of Cameroon has, through various circulars, encouraged the use of free and open source software (FOSS) in public administration and education to reduce licensing costs and promote technological sovereignty. Many schools use Ubuntu or Linux Mint in computer labs. However, proprietary software (Windows, Microsoft Office) remains widespread due to teacher familiarity, curriculum requirements, and compatibility with external examination formats.

8.5 I check my knowledge

EXERCISE 6 **MCQ: choosing the right category**

For each question, choose the single correct answer.

1. Which of the following is system software?
   (a) Microsoft Excel
   (b) A printer driver
   (c) VLC media player
   (d) Adobe Photoshop
2. Software distributed free of charge for a trial period, after which the user must pay to continue using it, is called:
   (a) freeware
   (b) open source software
   (c) shareware
   (d) bespoke software
3. Software written specially for one particular organisation, according to its own specifications, is:

(a) off-the-shelf software

(b) bespoke (custom-made) software

(c) utility software

(d) firmware

4. The BIOS of a computer is best classified as:

(a) application software

(b) firmware stored on a chip of the motherboard

(c) a general-purpose application

(d) shareware

EXERCISE 7 **True or false, with justification**

State whether each of the following statements is true or false, and justify your answer in one or two sentences.

1. "Freeware can be modified and redistributed freely by its users."
2. "An antivirus program is application software."
3. "A device driver is system software."
4. "Ubuntu Linux is proprietary software because it is an operating system."
5. "A compiler belongs to system software."

EXERCISE 8 **Key definitions**

Define each of the following terms clearly, and give one example of software for each:

1. system software;
2. application software;
3. utility software;
4. open source software;
5. off-the-shelf software.

EXERCISE 9 **Matching exercise**

Copy the table below and match each software product in column A to the most precise category in column B. Each category may be used once, more than once, or not at all.

No	Column A (software)	No	Column B (category)
1	Windows 11	a	Operating system
2	A keyboard driver	b	Utility software
3	WinRAR	c	Device driver
4	MS Word	d	General-purpose application
5	Ubuntu	e	Bespoke application
6	A payroll program written for one company	f	Open source operating system
7	A disk defragmenter	g	Shareware
8	The BIOS	h	Proprietary operating system
9	VLC media player	i	Firmware
10	Adobe Acrobat Reader	j	Freeware

8.6 I practise

EXERCISE 10 **Classifying real software**

Classify each of the following software products by role (system / application), by purpose (operating system, utility, driver, general-purpose application, bespoke application, firmware) and by licence (proprietary, freeware, shareware, open source), justifying each answer in one sentence:

1. Windows 11;
2. WinRAR;
3. Ubuntu;
4. MS Word;
5. VLC media player;
6. a printer driver;
7. Adobe Acrobat Reader;
8. the BIOS.

Present your answer in a table with four columns: Software, Role, Purpose, Licence.

EXERCISE 11 Choosing software for a school

The computer laboratory of a Government High School in Bamenda has 30 computers running Ubuntu. For each of the following needs, state whether the school requires system software or application software, and propose one concrete example, preferably free or open source:

1. protecting the computers against viruses;
2. typing and printing examination questions;
3. compressing students' project files to save disk space;
4. managing the startup of the machines and the user sessions;
5. drawing diagrams for Computer Science lessons.

EXERCISE 12 Licence analysis

A small business in Yaoundé downloads three programs from the Internet:

- Program A: free to download and use forever, but the source code is not available and modification is forbidden.
- Program B: free for 30 days; after that, a licence key costing money is required.
- Program C: free to download, and the licence explicitly allows users to study, modify and redistribute the source code.

1. Identify the licence type of each program (freeware, shareware or open source), justifying your answer.
2. For each program, state one advantage and one limitation for the business.
3. Which program would you recommend for a long-term, budget-limited use? Justify.

EXERCISE 13 Building a classification scheme

Draw a tree diagram (hierarchical chart) that classifies software into its main categories studied in this chapter: system software (with its sub-categories) and application software (with its sub-categories). Place at least one concrete software example at each leaf of your tree. Then add a second, independent classification based on usage rights (proprietary, freeware, shareware, open source) and explain in three sentences why these two classifications are independent of each other.

8.7 I prepare for the GCE

EXERCISE 14 Structured question: the cybercafé

A cybercafé owner in Douala wants to equip 10 new computers.

1. Distinguish system software from application software, giving one example of each. (4 marks)
2. Propose one operating system and two application programs suitable for a cybercafé, justifying each choice. (6 marks)
3. Explain to the owner two advantages of open source software for his business. (4 marks)
4. The owner also needs a program to clean temporary files and manage the disks. State the category of such a program and justify why it is not classified as a general-purpose application. (2 marks)
5. State two risks the owner faces if he installs pirated copies of proprietary software. (4 marks)

(Total: 20 marks)

EXERCISE 15 Scenario analysis: the hospital

A regional hospital in the North-West Region wants software to manage its patient records (registration, consultations, prescriptions, billing).

1. Explain the difference between off-the-shelf software and bespoke (custom-made) software. (4 marks)
2. Give two arguments in favour of buying an off-the-shelf hospital management package. (4 marks)
3. Give two arguments in favour of ordering bespoke software developed specifically for this hospital. (4 marks)
4. The hospital's computers will also need an operating system and device drivers. Explain why these two types of software are indispensable, whatever management solution is chosen. (4 marks)
5. Conclude by recommending one of the two options for this hospital, justifying your recommendation with two clear reasons. (4 marks)

(Total: 20 marks)

EXERCISE 16 **Essay: proprietary versus open source software**

"For Cameroonian public schools, open source software is always a better choice than proprietary software."
Discuss this statement. Your essay should:

1. define proprietary software and open source software, with one example of each; (4 marks)
2. present at least three advantages of open source software for public schools (cost, freedom, adaptation to local needs); (6 marks)
3. present at least two disadvantages or limitations of open source software in the Cameroonian school context (for example: technical support, training of teachers, compatibility); (4 marks)
4. present at least two situations in which proprietary software may be preferable; (4 marks)
5. end with a balanced personal conclusion. (2 marks)

(Total: 20 marks)

9 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — MCQ: choosing the right category**

1. Answer: (b) A printer driver. A device driver allows the operating system to communicate with a hardware device; it is system software. Excel, VLC and Photoshop all serve user tasks and are application software.
2. Answer: (c) shareware. Shareware is distributed free for a trial period or with limited functionality; continued use requires payment. Freeware is free forever, open source concerns freedom over the source code, and bespoke refers to custom development, not to a licence.
3. Answer: (b) bespoke (custom-made) software. It is developed specifically for one organisation according to its own specifications, unlike off-the-shelf software which is mass-produced.
4. Answer: (b) firmware stored on a chip of the motherboard. The BIOS/UEFI is low-level software permanently stored in ROM/flash memory; it runs at power-on, performs the POST and loads the operating system.

SOLUTION **Exercise 2 — True or false, with justification**

1. False. Freeware is free of charge but closed source: the copyright holder retains all rights, and modification and redistribution are normally forbidden. Only open source licences grant the freedoms to study, modify and redistribute.
2. False. An antivirus is a utility, hence system software: it acts on the system infrastructure (files, disks, security) rather than serving an end-user task such as writing or drawing.
3. True. A device driver enables the operating system to communicate with a specific hardware device; it operates and extends the capabilities of the hardware, which is precisely the role of system software.
4. False. The role (operating system) and the licence are independent classifications. Ubuntu is distributed under open source licences (mainly the GPL); its source code can be studied, modified and redistributed.
5. True. A compiler is a language translator; translators (compilers, interpreters, assemblers, linkers) are classified as system software because they are essential tools of the software development environment.

SOLUTION **Exercise 3 — Key definitions**

1. System software: the set of programs designed to operate, control and extend the processing capabilities of the hardware and to provide the platform on which applications run. Example: Windows 11 (operating system).
2. Application software: programs designed to help users perform specific tasks unrelated to the management of the computer itself. Example: Microsoft Word (word processing).
3. Utility software: a system program that performs maintenance, optimisation or security tasks on the computer system. Example: an antivirus, or 7-Zip for file compression.
4. Open source software: software distributed with a licence granting the freedoms to run, study, modify and redistribute the program and its source code. Example: LibreOffice.
5. Off-the-shelf software: a general-purpose, mass-produced application sold or distributed ready-made to a wide audience. Example: Adobe Photoshop.

SOLUTION **Exercise 4 — Matching exercise**

No	Software	Match	Justification
1	Windows 11	h	Proprietary OS: source code closed, licence paid to Microsoft.
2	Keyboard driver	c	Device driver: lets the OS communicate with the keyboard.
3	WinRAR	g	Shareware: free trial, then a paid licence key is required.
4	MS Word	d	General-purpose application: mass-produced word processor.
5	Ubuntu	f	Open source operating system (GPL and other free licences).
6	Payroll program for one company	e	Bespoke application: written to that company's specifications.
7	Disk defragmenter	b	Utility: maintenance task on the system's disks.
8	BIOS	i	Firmware: low-level code stored on a motherboard chip.
9	VLC media player	d (open source)	General-purpose application; also open source (GPL).
10	Adobe Acrobat Reader	j	Freeware: free of charge, closed source, no modification.

Marking note: category (a) "Operating system" is less precise than (f) or (h) for items 1 and 5; the instruction asks for the most precise category, so 1–h and 5–f are the expected answers. VLC (item 9) is best matched to (d) as a general-purpose application, with the remark that it is open source.

SOLUTION **Exercise 5 — Classifying real software**

Software	Role	Purpose	Licence
Windows 11	System	Operating system: manages hardware and runs other programs	Proprietary
WinRAR	System	Utility: compresses files, a maintenance/storage task	Shareware
Ubuntu	System	Operating system: manages resources and user sessions	Open source
MS Word	Application	General-purpose application: word processing for end users	Proprietary
VLC media player	Application	General-purpose application: plays audio/video for users	Open source
Printer driver	System	Device driver: lets the OS control the printer	Proprietary (usually)
Acrobat Reader	Application	General-purpose application: reads PDF documents	Freeware
BIOS	System	Firmware: boots the machine, POST, loads the OS	Proprietary (usually)

Justifications (one sentence each): Windows 11 and Ubuntu control the hardware and provide the platform for other programs, so they are system software; WinRAR acts on files and storage, hence a utility; Word, VLC and Acrobat Reader serve direct user tasks, hence applications; the printer driver translates OS commands for one device; the BIOS is stored on a ROM chip and starts the machine, hence firmware. For licences: Microsoft and Adobe products are closed source (paid for Word/Windows, free of charge for Reader); WinRAR requires payment after the trial; Ubuntu and VLC publish their source code under free licences.

SOLUTION **Exercise 6 — Choosing software for a school**

1. Protecting against viruses — system software (utility). Example: ClamAV, a free and open source antivirus available for Ubuntu.
2. Typing and printing examination questions — application software. Example: LibreOffice Writer, a free open source word processor installed by default on Ubuntu.
3. Compressing project files — system software (utility). Example: 7-Zip (p7zip) or the built-in Ubuntu archive manager, both free/open source.

4. Managing startup and user sessions — system software (operating system). This is the role of the OS itself: Ubuntu (with its bootloader GRUB and login manager) already fulfils it.
5. Drawing diagrams for lessons — application software. Example: LibreOffice Draw or GIMP/Inkscape, all free and open source.

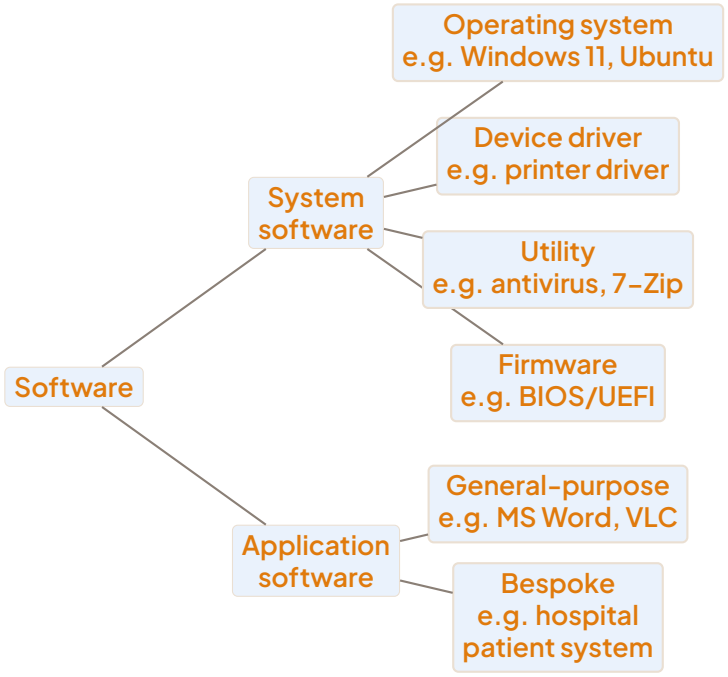
Key idea: needs 1, 3 and 4 concern the management and maintenance of the computer system itself (system software), while needs 2 and 5 serve direct user tasks (application software).

SOLUTION **Exercise 7 — Licence analysis**

1. Identification.
   - Program A = freeware: free of charge forever, but closed source and modification forbidden — the copyright holder keeps all rights.
   - Program B = shareware: free only for a 30-day trial, after which a paid licence key is required.
   - Program C = open source: the licence explicitly grants the freedoms to study, modify and redistribute the source code.
2. Advantage and limitation.
   - A (freeware): advantage — no cost, immediately usable; limitation — no adaptation possible, and the business depends entirely on the publisher for updates.
   - B (shareware): advantage — allows testing before buying, reducing the risk of a bad purchase; limitation — a recurring or one-off cost after 30 days, and work may be blocked if payment is delayed.
   - C (open source): advantage — free of charge, adaptable to the exact needs of the business, no dependence on a single vendor; limitation — may require technical skills to install, customise and maintain, and support is often community-based.
3. Recommendation. For long-term, budget-limited use, Program C (open source) is recommended: it costs nothing, can be legally used forever, and can be adapted as the business grows. Program A is acceptable as a second choice, but the business remains locked to the publisher's decisions, whereas C guarantees independence.

SOLUTION **Exercise 8 — Building a classification scheme**

Classification by role and purpose:



Hierarchical classification of software by role and purpose, with an example at each leaf.

Independent classification by usage rights:

Licence	Key feature	Example
Proprietary	Closed source, paid licence to use	Windows 11, MS Office
Freeware	Free of charge, closed source	Adobe Acrobat Reader
Shareware	Free trial, then payment required	WinRAR
Open source	Freedom to run, study, modify, redistribute	Ubuntu, LibreOffice

Independence of the two classifications (three sentences): The first classification describes what the software does in the machine (its role and purpose), while the second describes the legal conditions of its use, which is a completely different question. Consequently, the same role can appear under any licence: Ubuntu and Windows 11 are both operating systems, yet one is open source and the other proprietary. Knowing a program's category therefore tells us nothing about its licence, and vice versa — each product must be placed independently in both schemes.

SOLUTION **Exercise 9 — Structured question: the cybercafé**

Indicative mark scheme (20 marks).

1. (4 marks) System software operates and controls the hardware and provides the platform for other programs (1), e.g. Windows 11 or Ubuntu (1). Application software helps the user perform a specific task unrelated to managing the machine (1), e.g. a web browser or a word processor (1).

2. (6 marks) Operating system: Ubuntu — free of charge, no licence cost for 10 machines, resistant to most Windows viruses (2). Applications (any two, 2 marks each): Firefox/Chromium for browsing (essential service of a cybercafé, free) (2); LibreOffice for customers who need to type and print documents (free, opens MS Office files) (2). Examiner expectation: each choice must be justified by the cybercafé context, not merely named.

3. (4 marks) Any two advantages, 2 marks each: no licence fees, which drastically reduces the start-up cost for 10 computers (2); freedom to install on unlimited machines and to adapt the software, with no risk of licence violations (2).

4. (2 marks) It is a utility, i.e. system software (1), because it acts on the system infrastructure (disks, temporary files) rather than serving an end-user task like writing or drawing (1).

5. (4 marks) Any two risks, 2 marks each: legal prosecution and fines for copyright infringement (2); pirated copies often contain malware/viruses and receive no security updates, endangering customers' data (2). (Also acceptable: no technical support, unstable software.)

SOLUTION **Exercise 10 — Scenario analysis: the hospital**

Indicative mark scheme (20 marks).

1. (4 marks) Off-the-shelf software is mass-produced, ready-made and sold to many organisations for common needs (2); bespoke software is developed specifically for one organisation according to its own specifications and workflow (2).

2. (4 marks) Any two, 2 marks each: lower cost and immediate availability, so the hospital can start using it quickly (2); it has been tested by many users, so it is generally reliable and comes with vendor support and updates (2).

3. (4 marks) Any two, 2 marks each: it fits exactly the hospital's own procedures (registration, consultations, billing in the local context), avoiding useless or missing features (2); the hospital fully controls the features, the data and future modifications, which can give better confidentiality of patient records (2).

4. (4 marks) The operating system is indispensable because it manages the hardware,

the files and the user sessions, and no application can run without it (2). Device drivers are indispensable because without them the OS cannot communicate with the printers, network cards and other devices the hospital needs daily (2).

5. (4 marks) Example recommendation: bespoke software (1 justified choice), because patient-record management must match the hospital's exact workflow and language/billing practices (1.5), and because control over the code and data guarantees confidentiality and long-term independence from a vendor (1.5). Examiner expectation: either option is accepted provided the two reasons are clear, relevant to a hospital, and consistent with the choice.

SOLUTION **Exercise 11 — Essay: proprietary versus open source software**

Indicative mark scheme and model outline (20 marks).

1. Definitions (4 marks). Proprietary software: closed source code; the user buys only a licence to use and may not copy, modify or redistribute it — e.g. Microsoft Office (2). Open source software: licence granting the freedoms to run, study, modify and redistribute the program and its source code — e.g. Ubuntu Linux (2).

2. Advantages of open source for public schools (6 marks, 2 each).
   - Cost: no licence fees, a decisive argument for schools with very limited budgets; one system can be installed on all lab machines legally.
   - Freedom and sovereignty: the school is not tied to a vendor; software can be installed, copied and shared freely with teachers and students.
   - Adaptation to local needs: the source code can be studied (excellent for Computer Science teaching) and adapted or translated to local contexts.

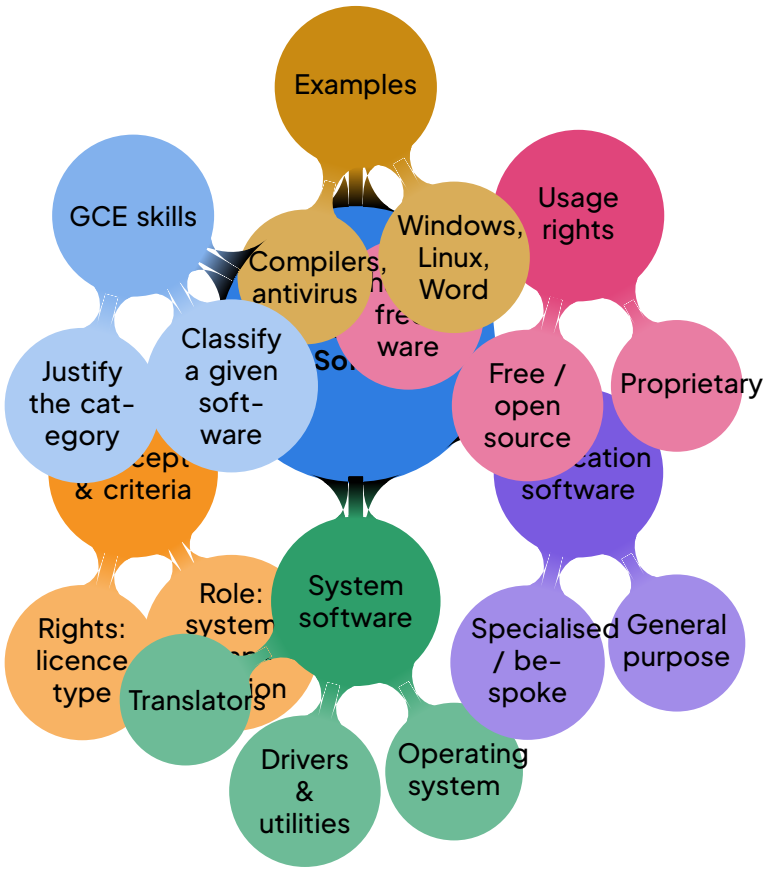
3. Limitations in the Cameroonian context (4 marks, 2 each).
   - Support and training: many teachers were trained on Windows/MS Office; switching requires training, and professional technical support for open source is scarce locally.
   - Compatibility: some examination formats, peripherals or commercial documents may require proprietary formats and drivers that work less smoothly under Linux.

4. Situations where proprietary software is preferable (4 marks, 2 each).
   - When a specific professional tool exists only in proprietary form (e.g. certain CAD or accounting packages required by a curriculum or an employer).
   - When guaranteed vendor support, training materials and compatibility with partners (ministries, examination boards, businesses) are critical.

5. Balanced conclusion (2 marks). Example: open source software offers major economic and pedagogical advantages and should be the default choice in public schools; however, the transition must be accompanied by teacher training and, where compatibility or support demands it, the measured use of proprietary software. The best policy is therefore not "always one or the other", but a reasoned combination guided by cost, needs and sustainability.

Examiner expectation: a structured essay (introduction, developed arguments with examples, personal but balanced conclusion), correct use of the chapter's vocabulary, and no confusion between freeware and open source.

10 SUMMARY SHEET



Mind map of the chapter: categories of software.

FICHE BILAN **L'essentiel à connaître pour l'examen**

Key definitions

- **Software**: the set of programs, procedures and data that tell the hardware what to do; the intangible part of a computer system (opposed to **hardware**).
- **Program**: an ordered sequence of instructions written in a programming language to perform a task.
- **System software**: software that manages and controls the hardware and provides a platform for other software (operating systems, device drivers, utilities, translators).
- **Operating system (OS)**: the master system software that manages the processor, memory, files, input/output devices and users (Windows, Linux, macOS, Android).
- **Device driver**: small system program allowing the OS to communicate with a specific

peripheral (printer, flash disk).

- **Utility**: maintenance tool of the system (antivirus, disk defragmenter, backup, compression).
- **Translator**: converts source code to machine code: **compiler** (whole program at once), **interpreter** (line by line), **assembler** (assembly language).
- **Application software**: software that performs a task for the end user (word processor, browser, school-fees management system).
- **General-purpose (off-the-shelf) software**: ready-made, used by many users (MS Word, Excel).
- **Specialised / bespoke software**: written for one specific organisation or task (a bank's or a hospital's custom package).

Classification by usage rights (licences)

- **Proprietary software**: paid licence, source code secret, copying forbidden (MS Windows, MS Office).
- **Free / open-source software**: source code available; freedom to use, study, modify, redistribute (Linux, LibreOffice, GIMP).
- **Freeware**: free of charge but source code closed (Skype, Adobe Reader).
- **Shareware**: free trial for a limited time or with limited features, then payment required (WinRAR).
- **Public domain**: no copyright restrictions at all.

Methods to master

- To classify a software: (1) ask who does it serve? — the machine/OS ⇒ system software; the end user ⇒ application software. (2) Then ask under which licence? — proprietary, open source, freeware, shareware.
- To justify in an exam: name the category, give the definition, then apply it to the example with one precise argument (role, availability of source code, cost).
- Distinguish the two classifications: by function (system/application) and by usage rights (licence). They are independent: Linux is system software and open source.

Common mistakes to avoid

- Confusing freeware (free of charge) with free software (freedom + source code): "free" has two meanings.
- Saying "the OS is an application": the OS is system software; it runs applications.
- Believing open source always means zero cost, or that proprietary always means paid: the licence concerns rights, not only price.

- Classifying an antivirus as application software: it is a utility, hence system software.
- Mixing compiler and interpreter: a compiler translates the whole program once; an interpreter translates and executes line by line.

GCE tips

- Typical question: "Classify the following software and justify": always give category + definition + justification for full marks.
- Memorise at least two Cameroon-relevant examples per category (e.g. a bespoke school-management package, LibreOffice in computer labs).
- Draw a clear two-axis table (function vs licence) in synthesis questions; it earns presentation marks.
- Define every key term in one precise sentence before discussing; vague definitions lose marks.

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS
OnBuch+